Chapter 1

Introduction

To develop this Machine Learning project we are going to use the CRISP-DM methodology, which is a well-known and widely used methodology for data mining, analytics, and data science projects. It is an iterative process that is composed of six phases.

In particular, the phases are:

1. Business Understanding: in this phase we will try to understand the problem and the objectives of the project. We will also try to understand the data that we have at our disposal and how we can use it to solve the problem.

2. Data Understanding: in this phase we will try to understand the data that we have at our disposal. We will try to understand the meaning of the data and how we can use it to solve the problem.

3. Data Preparation: in this phase we will try to prepare the data for the next phases. We will try to clean the data and to transform it in a way that will be useful for the next phases.

4. Modeling: in this phase we will try to build a model that will be able to solve the problem. We will try to find the best model for our problem.

5. Evaluation: in this phase we will try to evaluate the model that we have built. We will try to understand if the model is good enough to solve the problem.

6. Deployment: in this phase we will try to deploy the model that we have built. We will try to understand how we can use the model to solve the problems

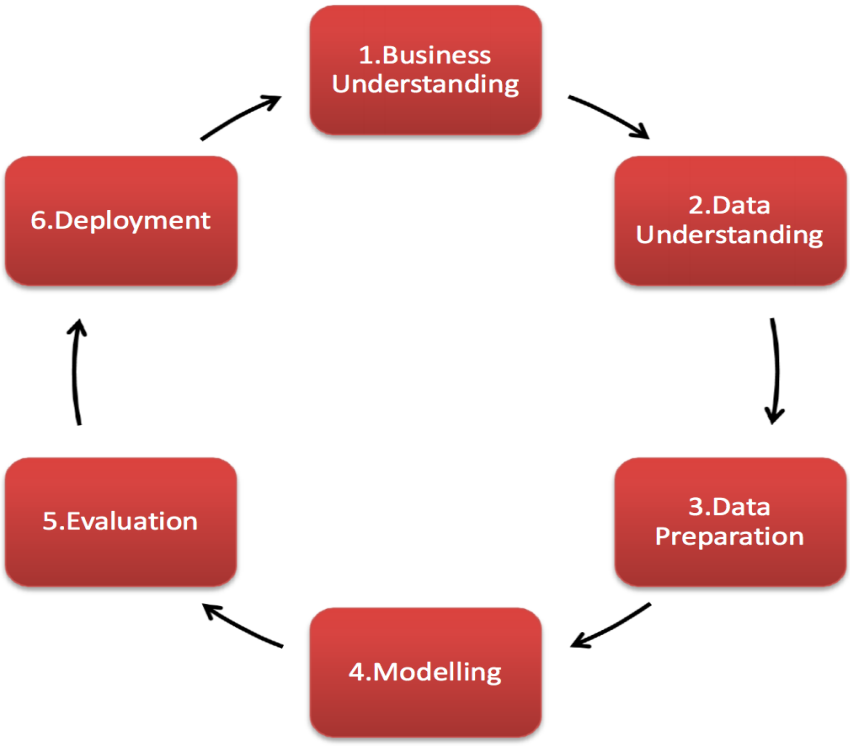


Figure 1.1: CRISP-DM methodology

Chapter 2

Business understanding

2.1 Background

Our project is about the Heart Disease and Stroke Prevention dataset

Given information about the public health burden of cardiovascular diseases in the United States, we aim to define an automatic procedure for categorizing these diseases. The primary goal of this project is to develop and validate a data-driven model that can accurately classify different types of cardiovascular diseases. By leveraging machine learning techniques, we intend to improve the efficiency and accuracy of disease categorization, which can ultimately aid in better diagnosis, treatment planning, and resource allocation in the healthcare system. This project not only addresses a critical public health issue but also demonstrates the potential of data science in transforming healthcare practices. Through rigorous testing and validation, we strive to ensure the reliability and effectiveness of the proposed mode

The dataset is Multivariate composed by 42640 rows and 29 columns, with target variable, the Topic.

The target has five categories:

- 'Heart Failure'
- 'Coronary Heart Disease'
- 'Stroke'
- 'Major Cardiovascular Disease'
- 'Diseases of the Heart (Heart Disease)'
- 'Acute Myocardial Infarction (Heart Attack)'

2.2 Business objectives

In this part we are going to analyze the business objective of the project. Our goal is to define an automatic procedure for categorizing cardiovascular diseases given information about the public health burden of Cardiovascular Diseases in the United States.

The success criteria of the project is to obtain a model that is able to categorize Cardiovascular Diseases with a good accuracy. In particular, we want to create a model that is able to categorize Cardiovascular Diseases with an accuracy of at least 0.8

2.4 Assessment of the situation

There is a very important aspect that we have to consider: there are a lot of variables in the dataset that contains a values of "none" especially in PriorityArea1, PriorityArea2, PriorityArea3, and Priority4 that could lead to thinking that the value has value. Actually, this is not true the "none" value means its missing value so we convert those value to "NAN". For this project, we are going to assume that the NAN value is missing value

2.5 Inventory of resources

For this project, we are going to use the following resources:

- Python 3.10.6: The programming language used for the project
- Jupyter Notebook: The IDE used for the project
- Pandas: used for data manipulation
- Numpy: used for data manipulation
- Matplotlib: used for data visualization
- Seaborn: used for data visualization
- Scikit-learn: used for machine learning

2.6 Project Plan

The development of the project will be divided into 5 main phases that are split like this:

- Phase 1: Data understanding (1 week)
- Phase 2: Data preparation (3 days)
- Phase 3: Modeling (1 week)
- Phase 4: Evaluation (4 days)
- Phase 5: Deployment (none)

Chapter 3

Data understanding

3.1 Data description

The dataset we are using, Heart Disease and Stroke Prevention, is a dataset composed of 42640 rows, and 29 columns, 28 of which are features and 1 is the target variable, the Topic.

3.2 Data exploration

The dataset contains 1236560 elements, 29 features, and 42640 rows. The attributes gives information about the Heart Disease, and the target variable is the Topic. The goal of this project develop and validate a data-driven model that can accurately classify different types of cardiovascular diseases the features are a mix of categorical and numerical variables.

	Year	LocationAbbr	LocationDesc	DataSource	PriorityArea1	PriorityArea2	PriorityArea3	PriorityArea4	Category	Topic	...	Break_Out_Category
0	2006	US	United States	Medicare	NaN	NaN	NaN	NaN	Cardiovascular Diseases	Heart Failure	...	Race
1	2005	US	United States	Medicare	NaN	NaN	NaN	NaN	Cardiovascular Diseases	Heart Failure	...	Race
2	2007	US	United States	Medicare	NaN	NaN	NaN	NaN	Cardiovascular Diseases	Coronary Heart Disease	...	Age
3	2008	US	United States	Medicare	NaN	NaN	NaN	NaN	Cardiovascular Diseases	Coronary Heart Disease	...	Gender
4	2004	US	United States	Medicare	NaN	NaN	NaN	NaN	Cardiovascular Diseases	Heart Failure	...	Overall
...	...	...	...	...	...	...	...	...	...	...	...	...
42635	2012	CO	Colorado	Medicare	Million Hearts	NaN	NaN	NaN	Cardiovascular Diseases	Stroke	...	Gender
42636	2010	MI	Michigan	Medicare	Million Hearts	NaN	NaN	NaN	Cardiovascular Diseases	Stroke	...	Gender
42637	2012	ME	Maine	Medicare	Million Hearts	NaN	NaN	NaN	Cardiovascular Diseases	Stroke	...	Race
42638	2013	ID	Idaho	Medicare	Million Hearts	NaN	NaN	NaN	Cardiovascular Diseases	Stroke	...	Race
42639	2012	IA	Iowa	Medicare	Million Hearts	NaN	NaN	NaN	Cardiovascular Diseases	Stroke	...	Gender

42640 rows × 29 columns

Activate W
Go to Settings

Figure 3.1

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42640 entries, 0 to 42639
Data columns (total 29 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Year                         42640 non-null  int64
 1   LocationAbbr                 42640 non-null  object
 2   LocationDesc                 42640 non-null  object
 3   DataSource                   42640 non-null  object
 4   PriorityArea1                9360 non-null   object
 5   PriorityArea2                0 non-null      float64
 6   PriorityArea3                14560 non-null  object
 7   PriorityArea4                0 non-null      float64
 8   Category                     42640 non-null  object
 9   Topic                        42640 non-null  object
 10  Indicator                    42640 non-null  object
 11  Data_Value_Type              42640 non-null  object
 12  Data_Value_Unit              42640 non-null  object
 13  Data_Value                   42111 non-null  float64
 14  Data_Value_Alt               42640 non-null  float64
 15  Data_Value_Footnote_Symbol   529 non-null    object
 16  Data_Value_Footnote          529 non-null    object
 17  LowConfidenceLimit           42111 non-null  float64
 18  HighConfidenceLimit          42111 non-null  float64
 19  Break_Out_Category           42640 non-null  object
 20  Break_Out                    42640 non-null  object
 21  CategoryId                   42640 non-null  object
 22  TopicId                      42640 non-null  object
 23  IndicatorID                  42640 non-null  object
 24  Data_Value_TypeID            42640 non-null  object
 25  BreakOutCategoryId           42640 non-null  object
 26  BreakOutId                   42640 non-null  object
 27  LocationID                   42640 non-null  int64
 28  GeoLocation                  41820 non-null  object
dtypes: float64(6), int64(2), object(21)
memory usage: 9.4+ MB
```

Fig 3.2

3.2.1 Preliminary operation on data preparation

In order to achieve better results looking at data and graphs, we decided to do some data transformation and cleaning before the data processing part. This choice brings an easier and more intuitive data visualization and analysis for us, in order to get a better insight of the data.

The action carried out in this part are the following:

1. Transforming “none” values into “NAN” for all features which has meaning of missing values.

2. We specified a threshold of 65% to drop an attribute if its missing percentage is greater than 65%

```
In [33]: # Which Columns are missing values and what is the extent of Missigness
         df.isnull().sum()/len(df)*100
```

```
Out[33]: Year                          0.000000
         LocationAbbr                  0.000000
         LocationDesc                  0.000000
         DataSource                    0.000000
         PriorityArea1                78.048780
         PriorityArea2               100.000000
         PriorityArea3                65.853659
         PriorityArea4               100.000000
         Category                      0.000000
         Topic                         0.000000
         Indicator                     0.000000
         Data_Value_Type               0.000000
         Data_Value_Unit               0.000000
         Data_Value                    1.240619
         Data_Value_Alt                0.000000
         Data_Value_Footnote_Symbol   98.759381
         Data_Value_Footnote          98.759381
         LowConfidenceLimit            1.240619
         HighConfidenceLimit           1.240619
         Break_Out_Category            0.000000
         Break_Out                     0.000000
         CategoryId                    0.000000
         TopicId                       0.000000
         IndicatorID                   0.000000
         Data_Value_TypeID             0.000000
         BreakOutCategoryId            0.000000
         BreakOutId                    0.000000
         LocationID                    0.000000
         GeoLocation                   1.923077
         dtype: float64
```

Fig 3.3

Here the dropped columns are:

- PriorityArea1
- PriorityArea2
- PriorityArea3
- PriorityArea4
- Data_Value_Footnot_Symbol
- Data_Value_Footnote

3. Drop Columns that do not vary

- DataSource which has Medicare values throughout
- Category column consists of cardiovascular diseases values entirely.
- Data_Value_Type

4. Removing unnecessary attributes for prediction

- GeoLocation
- Indicator

5. Dropped ID Variable not useful for Model

```
df=df.drop(['CategoryId','TopicId','IndicatorID','Data_Value_TypeID','BreakOutCategoryId','BreakOutId','LocationID'], axis=1)
```

From the dataset provided, the ID columns that could potentially be removed are:

- 'CategoryId'
- 'TopicId'
- 'IndicatorID'
- 'Data_Value_TypeID'
- 'BreakOutCategoryId'
- 'BreakOutId'
- 'LocationID'

6. Dropped “LocationAbbr” column which is abbreviation for LocationDesc

7. Filling the missing values for LowConfidenceLimit and HighConfidenceLimit using median, since these features are right-skewed

8. Changing all the columns with attribute object to category type

9. Changing the column “Year” with attribute int64 to category type

10. Dropped Data_Value Attribute since the correlation analysis shows Data_value and Data_Value_Alt are highly similar Additionally, Data_value shows a strong correlation with both HighConfidenceLimit and LowConfidenceLimit.

3.2.2 Data Exploration - More in depth

```
In [50]: df.describe().T
```

Out[50]:

	count	mean	std	min	25%	50%	75%	max
Data_Value	42111.0	14.780896	13.286487	0.4	4.5	10.7	21.6	333.3
Data_Value_Alt	42464.0	14.641397	13.318586	-2.0	4.4	10.5	21.6	333.3
LowConfidenceLimit	42464.0	12.624421	11.413099	0.0	3.8	8.2	19.5	94.2
HighConfidenceLimit	42464.0	17.037667	17.840095	1.1	5.1	12.7	23.3	710.5

Fig 3.4: Numerical info

```
In [51]: df.describe(include='category').T
```

Out[51]:

	count	unique	top	freq
Year	42464	10	2006	4252
LocationDesc	42464	52	Nebraska	820
Topic	42464	6	Heart Failure	19064
Data_Value_Unit	42464	2	Percent (%)	28080
Break_Out_Category	42464	4	Race	20624
Break_Out	42464	9	Female	5200

Fig 3.5: Categorical info

3.2.3 Plotting

Target variable

As it can be seen in the graph, the target variable is strongly unbalanced, and in particular the label ”Heart Failure” has very large data compared to the other ones. This will be taken into account during the modeling phase, considering also some oversampling techniques.

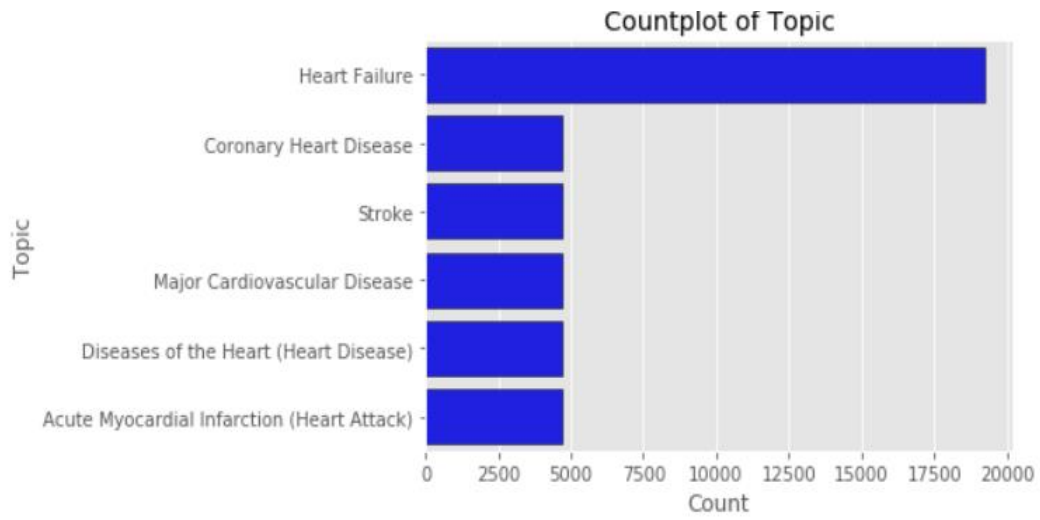


Fig 3.6

Features

We visualize the relative missingnes of columns to have better understanding,

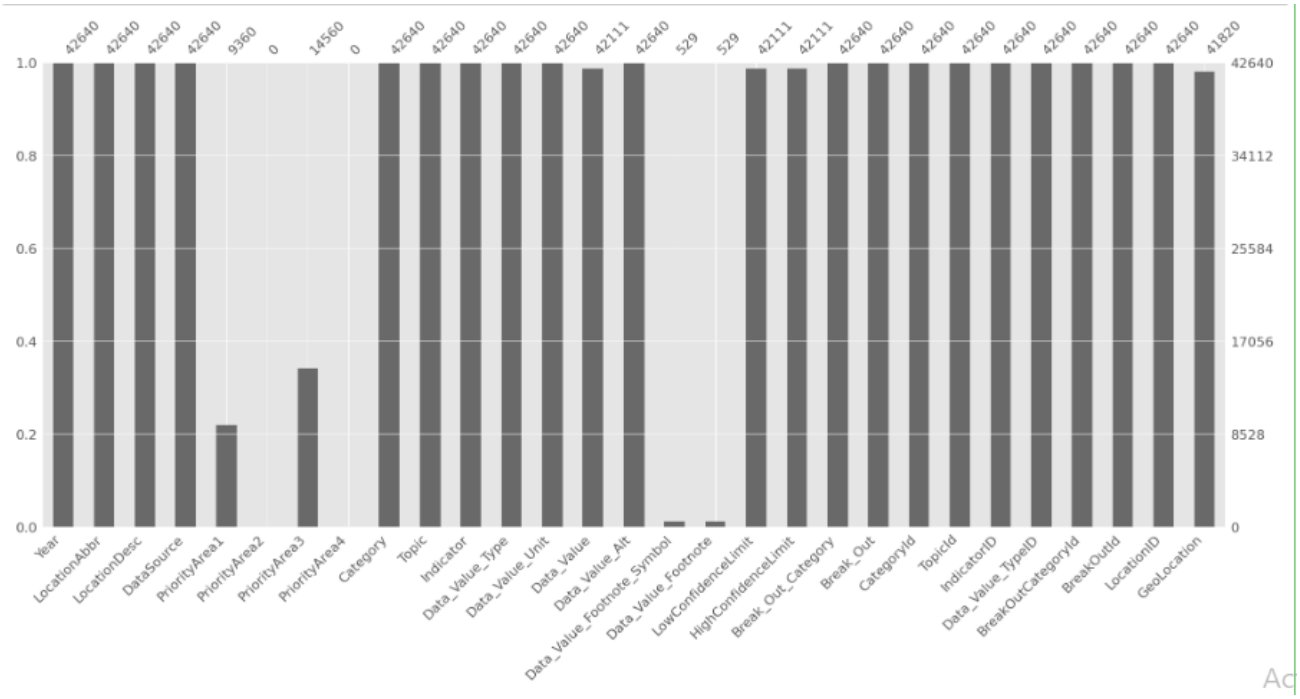


Fig 3.7

As we can see from the above figure there are total 86629 missing values in the given dataset.

From this, 529 Missing values are in Datavalue variable.42111 missing values each for Data value foot note symbol and Datavalue foot note. This can affect the performance of our model.529 missing values each for confidence limit low and confidence limit high respectively.820 missing values for Geolocation.

Data_Value_Footnot_Symbol, Data_Value_Footnote consists tremendous number of missing values and are not required for prediction.

It is advisable to drop them. Also GeoLocation is to be dropped as it is unnecessary for prediction

As we can see The figure below we visualize the plot of the PriorityArea1, PriorityArea2, PriorityArea3 and PriorityArea4 respectively to show their missing values.it indicates PriorityArea2,PriorityArea4 contains None values throughout, also these variables don't serve any purpose in prediction of our target variable. We dropped those variables.

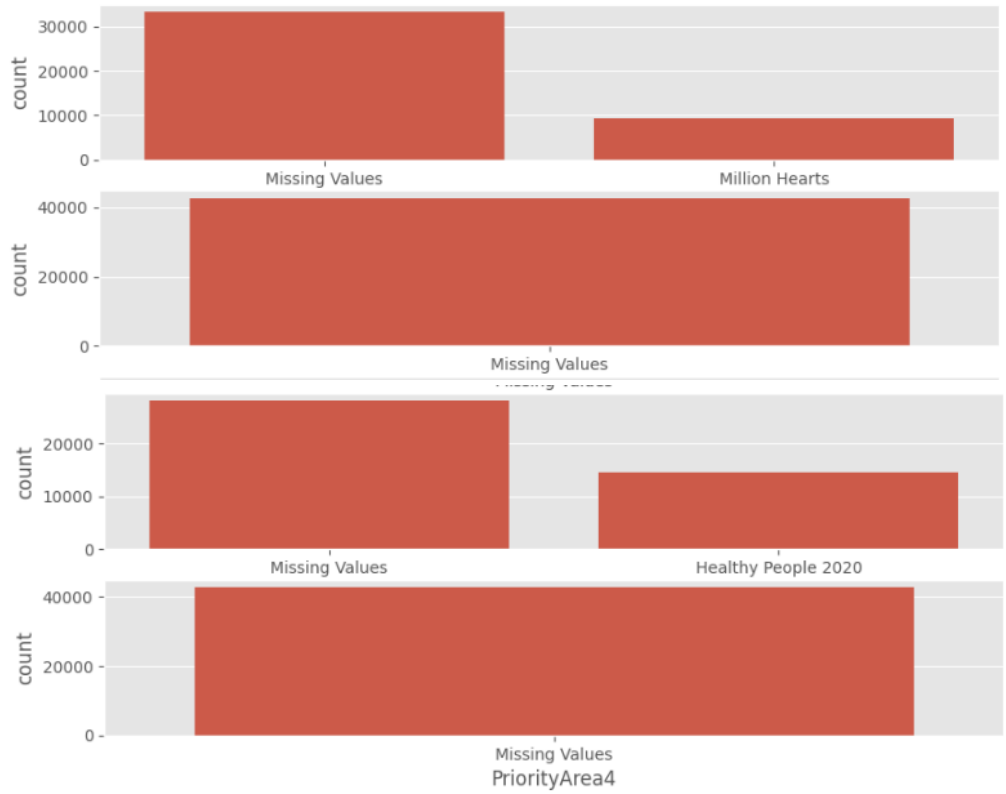
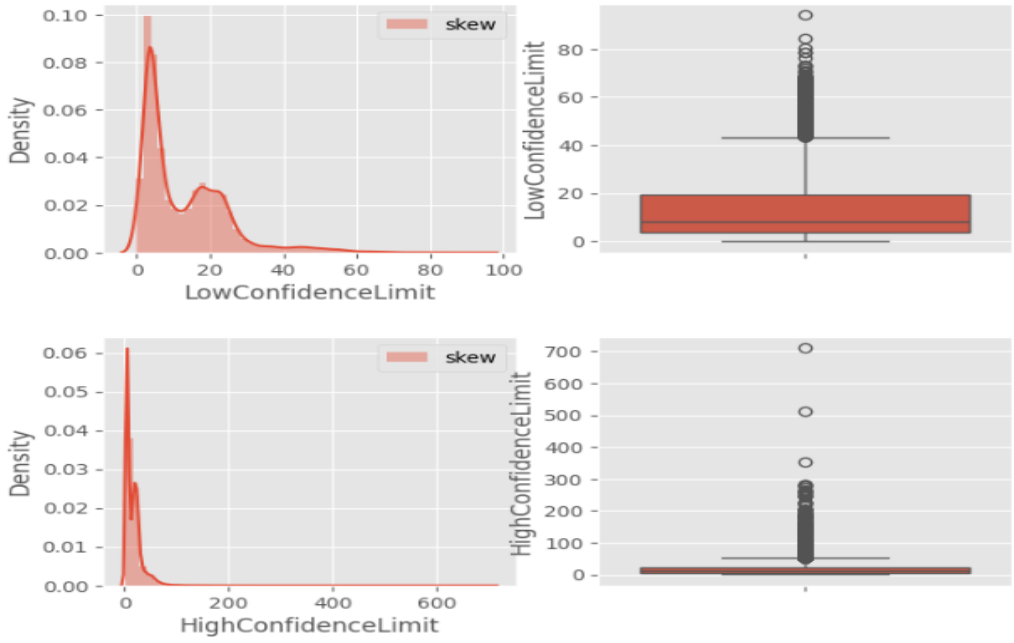


Fig 3.8

As we can see from the figure below for the attributes "LowConfidenceLimit" and "HighConfidenceLimit," which were right-skewed, missing values were filled using the median. This method is effective for skewed distributions as it is less affected by outliers compared to the mean.



Examples of useless feature

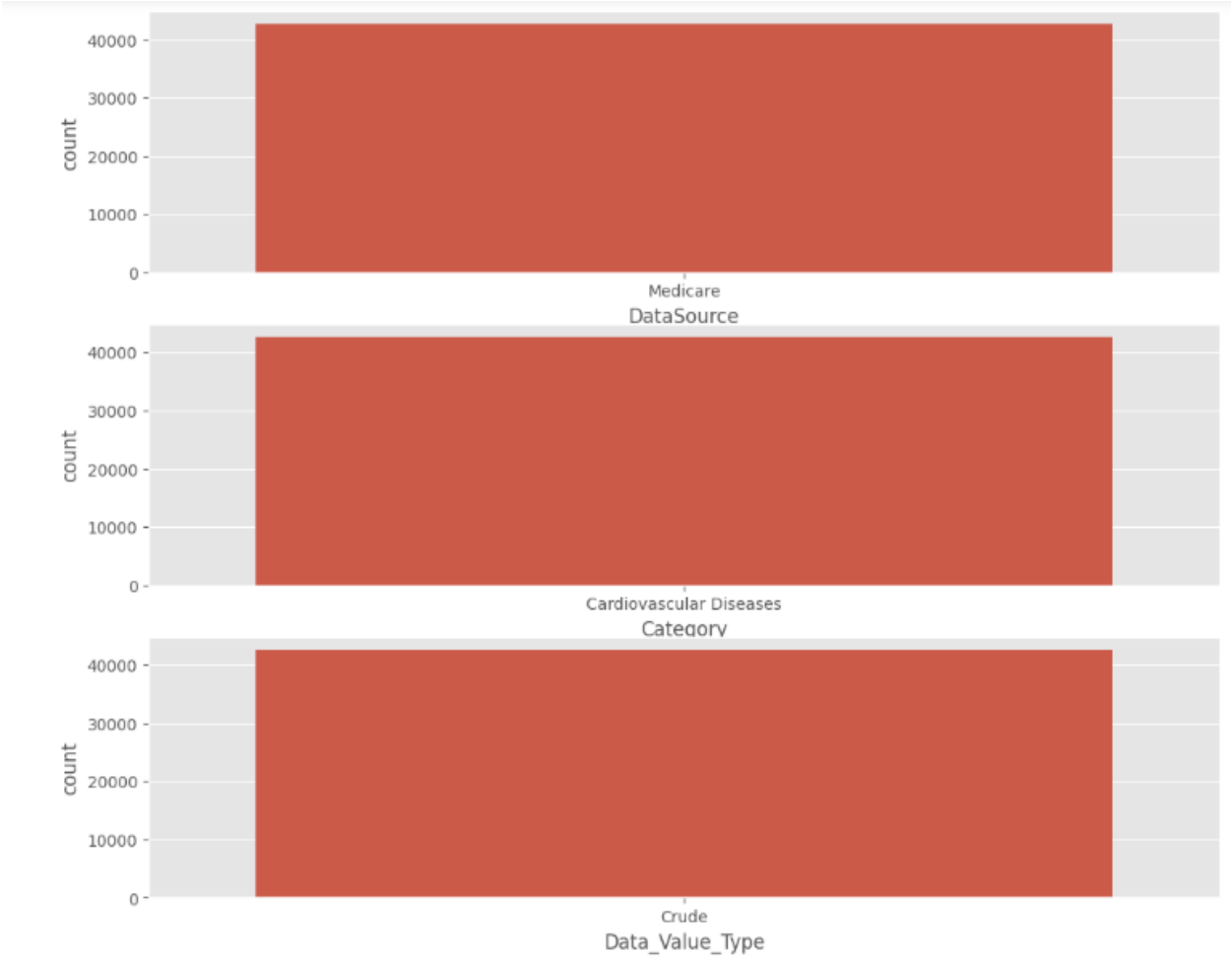


Fig 3.9

Since the features DataSource, Category, Data_Value_Type had only the 1 value for the 100% of the data, we decided to drop it.

Example of nice feature

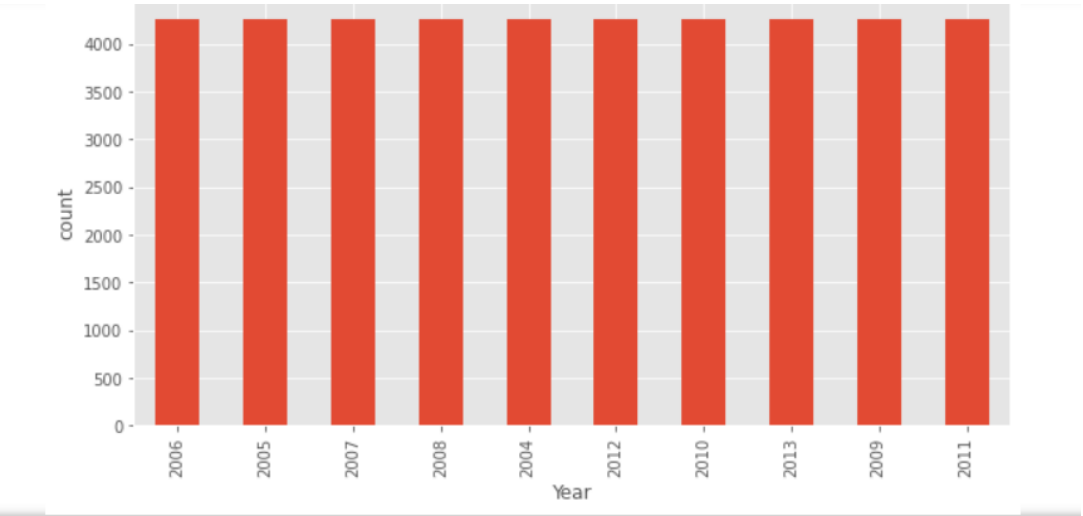


Fig 3.10 Year

The feature is well distributed. It has a good correlation with the target variable.

Example of Categorical feature correlated with the target variable

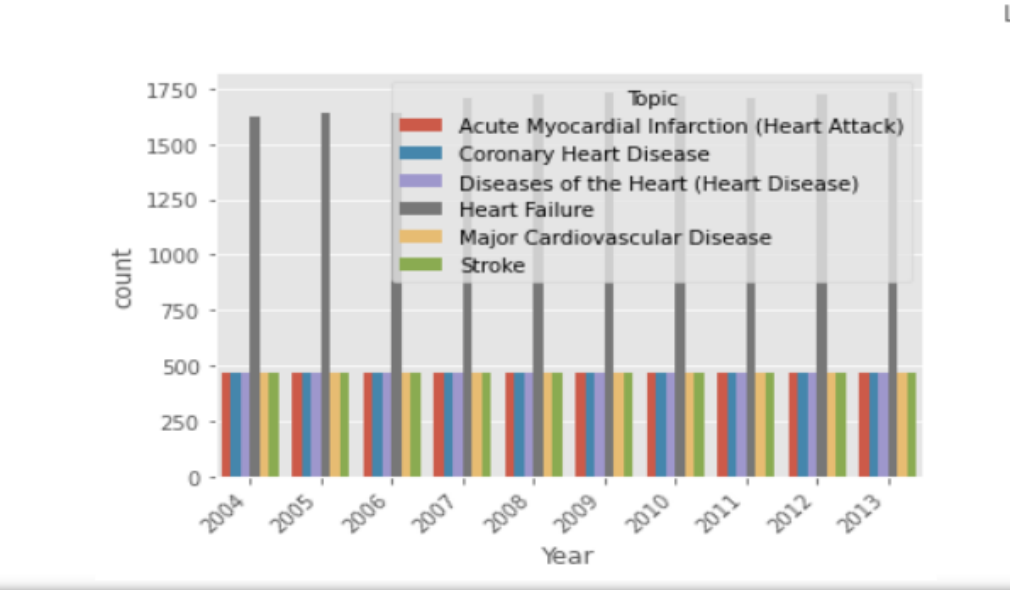


Figure 3.11: Year Bar Plot

From this graph we can see that the Year is a good feature to predict the target variable. In fact, the Heart Failure is very high as compared to others. This is a good feature to predict the target variable.

Correlation Matrix

The correlation matrix is a useful tool to understand the relationship between the variables. In particular, it is possible to see the correlation between the variables.

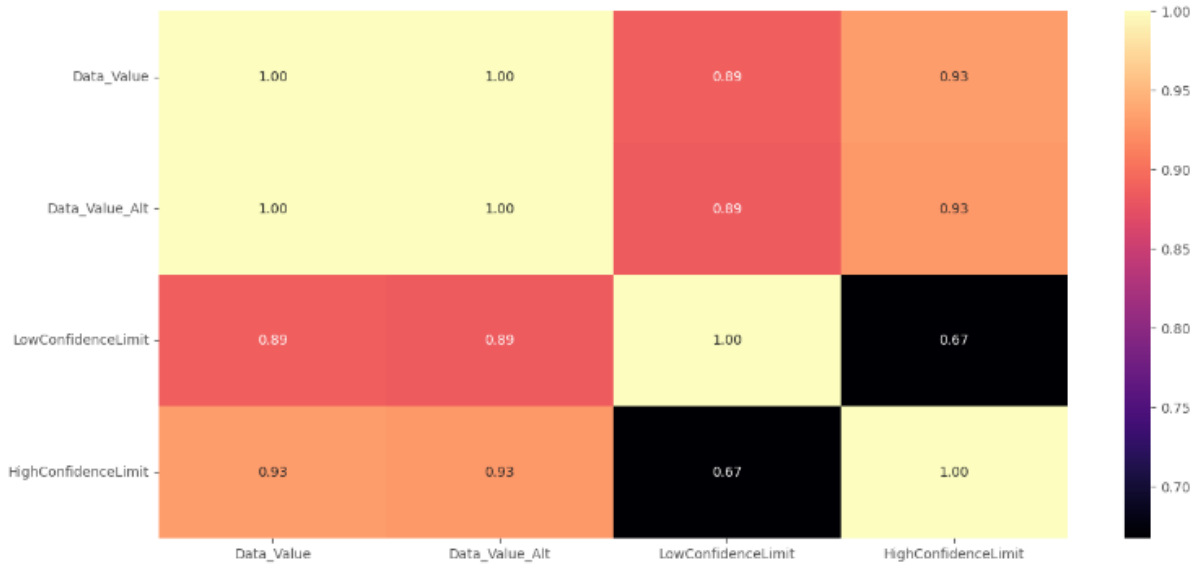


Figure 3.12: Correlation Matrix

In this correlation matrix we can see that some features have a direct correlation with other variables. We decided to remove some of these features. It is clear that Data_value and Data_Value_Alt are similar.

Also high correlation is seen between Data_value, HighConfidenceLimit and LowConfidenceLimit

In particular:

- Data_value 0.93 correlation with HighConfidenceLimit
- Data_value 0.89 correlation with LowConfidenceLimit
- Data_value 1.00 correlation with Data_Value_Alt

Chapter 4

Data preparation

4.1 Data pruning

As previously seen, many of the features were not encoding patterns, evenly distributed and significant into the specific domain, so we dropped them. This reduced the total amount of features, from the 29 starting ones, to 9.

List of features removed

Numerical: 'Data_Value',' PriorityArea2','PriorityArea4', 'LocationID'

Categorical: 'LocationAbbr',' DataSource',' PriorityArea1','PriorityArea3','Category',' Indicator', ' Data_Value_Type',' Data_Value_Footnote_Symbol', ' Data_Value_Footnote',' CategoryId','TopicId', IndicatorID','Data_Value_TypeID',' BreakOutCategoryId', ' BreakOutId',' GeoLocation',

4.2 Conversion, binarization, encoding

A variable, even if scaled on numbers, were encoding a categorical information. Specifically, a **Year** have been converted in to categorical object type.

In the process of preparing data for analysis, binarization and normalization are crucial steps to ensure that the dataset is in an optimal format for machine learning models. Here, we explain the steps taken to achieve these transformations using a sample code.

4.2.1 Binarization

Binarization refers to converting categorical variables into a binary format, often necessary for algorithms that require numerical input. In this example, binarization is achieved through ordinal and one-hot encoding.

1. Ordinal Encoding for 'Year': The 'Year' column, initially categorical, is converted into a numerical format using the OrdinalEncoder from sklearn.preprocessing. This encoding assigns a unique integer to each unique year, maintaining the temporal order.

```
from sklearn.preprocessing import OrdinalEncoder
ord_enc = OrdinalEncoder()
df_cleaned['Year'] = ord_enc.fit_transform(df_cleaned[['Year']])
```

2. One-Hot Encoding for Categorical Attributes: After identifying categorical attributes (excluding 'Topic'), one-hot encoding is applied. One-hot encoding converts each categorical value into a

new binary column (0 or 1), where the presence of a category is marked as 1 and absence as 0. This transformation ensures that the model can interpret categorical data without assuming any ordinal relationship.

```
cat_attributes = list(df_cleaned.select_dtypes(include=['category']).columns)
cat_attributes.remove('Topic')
df2 = pd.get_dummies(df_cleaned, columns=cat_attributes)
```

4.2.2 Normalization

Normalization scales numerical attributes to a specific range, typically 0 to 1, enhancing the performance and convergence speed of many machine learning algorithms.

1. Identifying Numerical Attributes: Numerical attributes (excluding binary columns from one-hot encoding) are identified. The 'Year' column, although encoded numerically, is also excluded from normalization to preserve its ordinal nature.

```
numerical_attributes = list(df2.select_dtypes(include=['int64', 'float64']).columns)
numerical_attributes.remove('Year')
```

2. Applying **RobustScaler**: it is an important preprocessing technique in machine learning, especially when dealing with real-world datasets containing outliers or non-normal distributions. It ensures that models are more robust and generalizable by scaling features using robust statistics, thereby preserving the integrity of the data and reducing the impact of outliers on model performance. Since the dataset contains outliers we selected this type of normalization

By implementing these steps, the data is transformed into a suitable format for further analysis and modeling. The resulting DataFrame, df2, now contains properly encoded categorical attributes and normalized numerical attributes, ensuring consistency and enhancing model performance.

We finally replace values in the 'Topic' column which is our target attribute with numerical values

- Heart Failure: 0
- Coronary Heart Disease: 1
- Stroke: 2
- Major Cardiovascular Disease: 3
- Diseases of the Heart (Heart Disease: 4
- Acute Myocardial Infarction (Heart Attack: 5

The final shape of our dataset, after all the data cleaning process, is df.shape (40329, 9)

Chapter 5

Modeling and model evaluation

5.1 Preliminary operations

First steps in this part of the modeling was to split our dataset in to train set, used to train every model, and a test set, to compute the evaluation metrics of them and further split the training set to training and validation set. Then, to avoid bias due to the unbalanced target classes, we did an oversampling over the sets using the SMOTE technique finally, we built multiple models, iterating over each one to optimize parameter settings for improved performance on the validation set. We selected the best-performing model based on its predictive accuracy and evaluated its generalization ability on the independent test set.

5.1.1 Decision Tree Classifier

A Decision Tree is a supervised learning model that employs a hierarchical structure of nodes to make decisions based on input features. Each internal node represents a feature, and the branches represent possible outcomes based on the feature's value. Decision Trees excel in capturing complex relationships and are easy to interpret. However, they are prone to overfitting and can struggle with handling continuous data or outliers.

5.1.2 Random Forest

The Random Forest algorithm is an ensemble learning method that combines multiple decision trees to make predictions. It generates a multitude of decision trees and aggregates their outputs to produce a final prediction. Random Forests are known for their robustness, versatility, and resistance to overfitting. They are particularly effective for handling high-dimensional data and are widely used for classification and regression tasks

5.1.3 Neural Network

A Neural Network is a computational model inspired by the human brain's structure, consisting of interconnected layers of nodes (neurons). It works by receiving input data, processing it through hidden layers where each neuron applies a transformation using weights and biases, and generating an output. For prediction and classification, the network learns from labeled training data by adjusting these weights through a process called backpropagation to minimize prediction errors. Once trained, the network can make predictions or classify new data based on the learned patterns. Neural Networks are particularly effective for complex, non-linear problems across various domains such as image recognition, natural language processing, and time series forecasting.

5.1.4 AdaBoost Classifier

Adaboost, short for Adaptive Boosting, is another ensemble learning method that sequentially combines weak learners to create a strong predictive model. It assigns higher weights to misclassified instances in each iteration, enabling subsequent models to focus on those instances. Adaboost is versatile, relatively easy to implement, and can be used for both classification and regression tasks. However, it is sensitive to noisy data and outliers.

5.1.5 Gaussian Naive Bayes Classifier

Gaussian Naive Bayes is a probabilistic classifier that applies Bayes' theorem with the assumption of independence between features. It assumes that the features follow a Gaussian distribution and calculates the probability of each class given the observed feature values. Despite its simplicity, Gaussian Naive Bayes can perform well in many real-world applications, especially when the independence assumption holds.

5.1.6 K-Neighborhood classifier

K-Nearest Neighbors is a non-parametric algorithm used for both classification and regression tasks. It classifies new instances by finding the closest labeled instances in the feature space and assigning the majority class or predicting the average value of their targets. KNN is easy to understand and implement, but it can be computationally expensive, especially with large datasets. It is also sensitive to the choice of the number of neighbors (K) and the distance metric.

5.2 Train and test splits and oversampling

The dataset used in this project was divided into an 80% training set and a 20% testing set and further divided the training set in to 80% training set and 20% validation set. This split ensured that a substantial portion of the data was dedicated to training the machine learning models, while also providing an independent subset for evaluating their performance. Furthermore, it was observed that the dataset exhibited class imbalance, where the minority class had significantly large instances compared to the majority class, **Heart Failure** class has very large data compared to the other ones. To address this issue and ensure fair representation of all classes during model training, oversampling techniques were applied. Specifically, oversampling was performed for the minority class using the BorderlineSMOTE method.

5.2.1 BorderlineSMOTE

BorderlineSMOTE generates synthetic samples for the minority class, effectively increasing its representation in the dataset. By incorporating these synthetic samples, the training data became more balanced, allowing the models to learn from both the majority and minority classes more effectively. By splitting the data into training and testing sets and applying oversampling techniques, we aimed to improve the robustness and performance of the machine learning models, particularly in handling class imbalance challenges. The resulting datasets were then used to train and evaluate the models, providing a comprehensive assessment of their effectiveness in addressing the problem at hand.

5.2 Model Evaluation

5.2.1 Comparison and Selection of Best model

The Neural Network showed best result across accuracy, precision, recall, and F1-score among all models. This model demonstrated balanced and robust performance with relatively few misclassification.

Random Forest also showed a better accuracy, precision, recall, and F1-score compared to the C45 and CART Decision Tree which both have almost similar good results.

The AdaBoostClassifier model achieved the lowest accuracy among all models, indicating a lower overall correctness in its predictions. Gaussian Naïve Bayes Classifier models exhibited a performance with reasonably moderate accuracy, precision, recall, and F1-score. In contrast, the K-nearest Neighbors model had lower accuracy compared to most other models.

- Based on the information provided above we selected our best model **Neural Network** which has the following result.

```
Training loss did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
Classification metrics:
              precision    recall  f1-score   support

           0       0.95      0.93      0.94      3009
           1       0.74      0.90      0.81       705
           2       0.82      0.72      0.77       726
           3       0.97      0.97      0.97       762
           4       0.96      0.97      0.97       770
           5       0.88      0.88      0.88       690

    accuracy                           0.91      6662
   macro avg       0.89      0.89      0.89      6662
weighted avg       0.91      0.91      0.91      6662
```

- Next we will evaluate Neural Network model on test set and measure its accuracy

5.2.2 Predictions on test set

```
              precision    recall  f1-score   support

           0       0.94      0.93      0.93      3734
           1       0.79      0.79      0.79       952
           2       0.71      0.73      0.72       844
           3       0.97      0.98      0.97       930
           4       0.98      0.96      0.97       912
           5       0.86      0.87      0.86       956

    accuracy                           0.90      8328
   macro avg       0.87      0.88      0.87      8328
weighted avg       0.90      0.90      0.90      8328
```

```
[48]: print(f'Accuracy: {accuracy_score(y_test, predictions):.2f}')
      Accuracy: 0.90
```

Figure5.1TestAccuracy

As we can see from the above figure our model Neural Network achieved an accuracy of 0.90 which is a good result

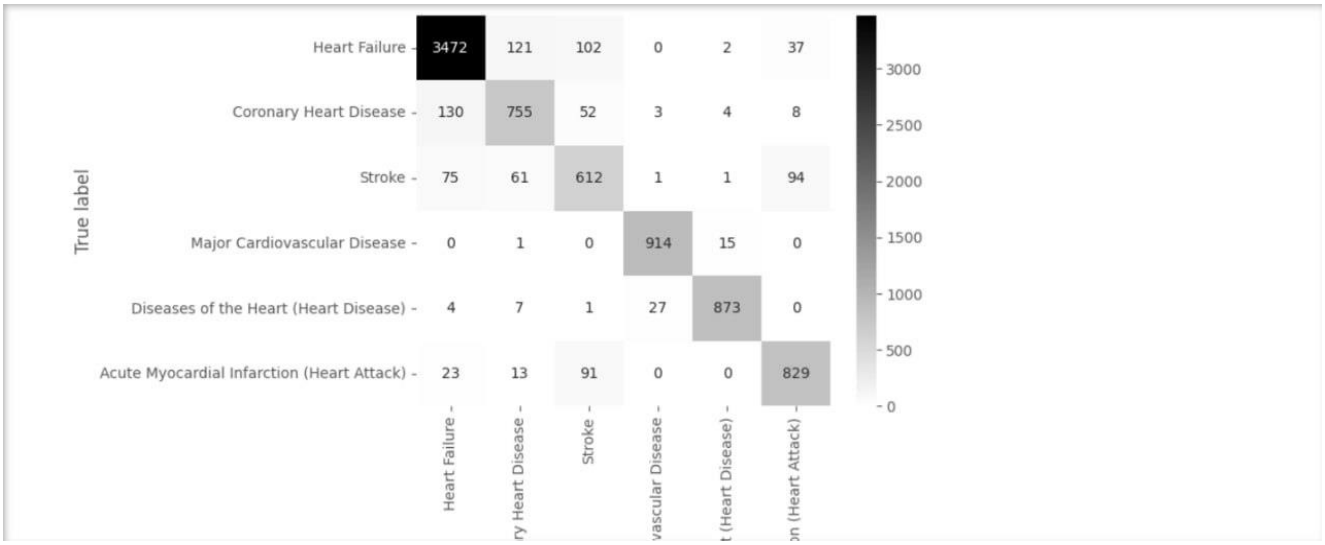


Figure5.2 Confusion Matrix

The above figure shows the confusion matrix which evaluates the performance of our model (Neural Network) for various cardiovascular diseases. Key points include:

High Accuracy: The model performs well for:

- Heart Failure (3472 correct)
- Major Cardiovascular Disease (914 correct)
- Diseases of the Heart (873 correct)
- Acute Myocardial Infarction (829 correct)

Common Misclassifications:

- Heart Failure often misclassified as Coronary Heart Disease (121) and Stroke (102)
- Coronary Heart Disease frequently misclassified as Heart Failure (130) and Stroke (52)
- Color Scale: Darker colors indicate higher frequencies of predictions.

Summary: The model accurately classifies most cases but struggles with distinguishing between similar conditions like Heart Failure and Coronary Heart Disease.

5.3 K-fold cross-validation

KFold is a technique used in machine learning for cross-validation. It involves splitting the dataset into k equal-sized folds or subsets. The model is then trained and evaluated k times, each time using a different

fold as the validation set and the remaining folds as the training set. This ensures that each data point is used for both training and validation, providing a more reliable estimation of the model's performance. Cross val score is a function provided by the scikit-learn library that automates the process of performing KFold cross-validation and computing evaluation metrics. It simplifies the process by allowing you to specify the number of folds (k) and the evaluation metric of interest, such as accuracy or mean squared error. The function takes care of splitting the data, training the model, and evaluating its performance on each fold. It returns an array of scores, representing the performance metric for each fold. In our project, we utilized KFold cross-validation along with cross val score to assess the performance of our machine learning models. By employing this technique, we were able to obtain a more robust evaluation of the models, taking into account variations in performance across different subsets of the data. This helped us gain a better understanding of how well our models generalize to unseen data and allowed us to make more informed decisions about their effectiveness.

- ❖ We increased our model Accuracy using K-Fold cross validation

[19]:

	C45	CART	RF	NeuralNetwork	GB	AB	NB	KNN
0	0.894964	0.894686	0.914580	0.931831	0.886060	0.466750	0.755704	0.685448
1	0.895381	0.894546	0.918058	0.929327	0.888703	0.695604	0.753756	0.689761
2	0.899137	0.900390	0.919727	0.928909	0.887730	0.484836	0.754730	0.688370
3	0.895506	0.897315	0.917907	0.930847	0.886044	0.428830	0.761375	0.689857
4	0.892723	0.893280	0.911785	0.932656	0.887297	0.510366	0.764436	0.687213

Figure 5.3: K-fold cross-validation

5.3 ROC Curve

The ROC curve is like a graph that shows how well a machine learning system can tell things apart. It plots the true positive rate (how well it detects the right things) against the false positive rate (how often it makes mistakes) as we change the threshold for making decisions. The curve shows how the sensitivity (ability to detect) changes as the fall-out (making mistakes) varies. Basically, if we know the probabilities of detecting something and making a mistake, we can create the ROC curve by looking at the cumulative probabilities. We plot the detection probability on the y-axis and the false-alarm probability on the x-axis. It helps us see how well a classifier can distinguish between positive and negative cases. We can look at the shape of the curve and the area under it to evaluate the performance. A larger area under the curve means a better classifier that can tell things apart more accurately. So it is a handy way to measure how well a classifier can do its job. It lets us compare different models and see how good they are at separating the good from the bad.

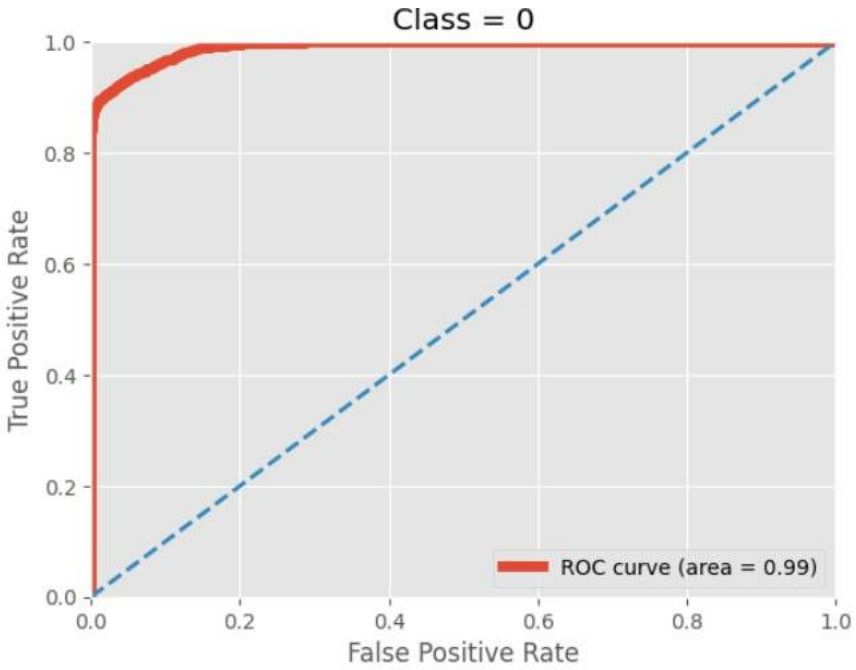


Figure 5.5: ROC Curve of Neural Network – 0

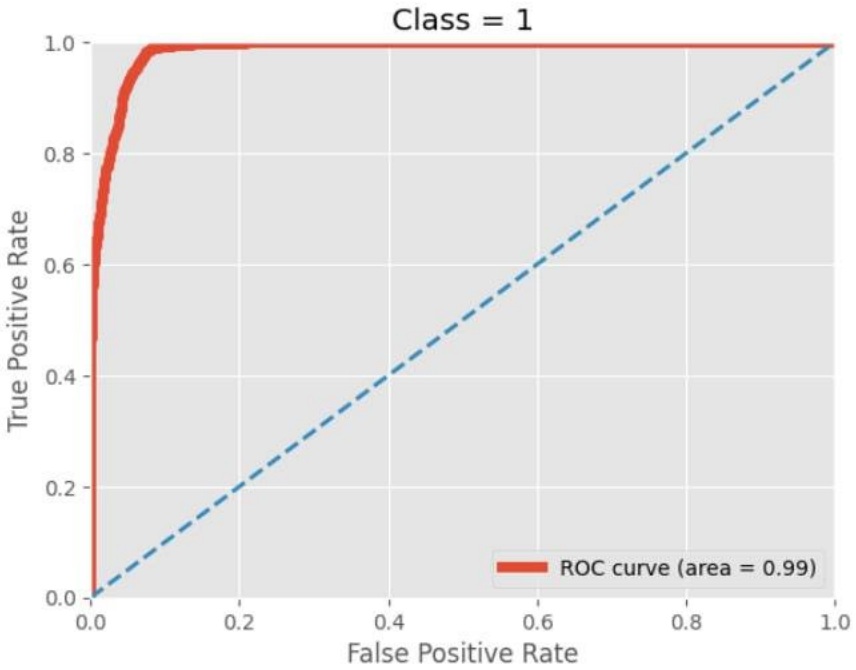


Figure 5.5: ROC Curve of Neural Network– 1

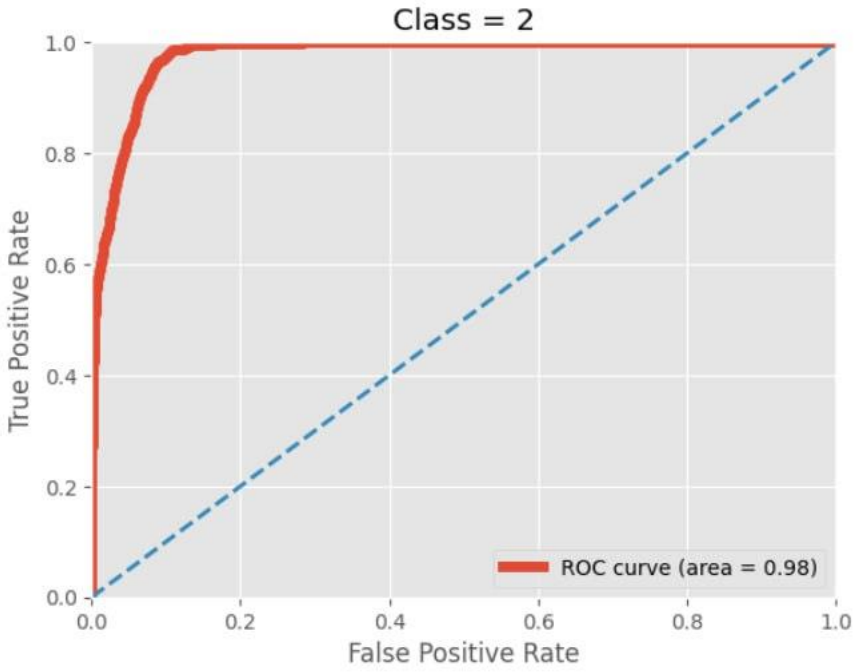


Figure 5.5: ROC Curve of Neural Network – 2

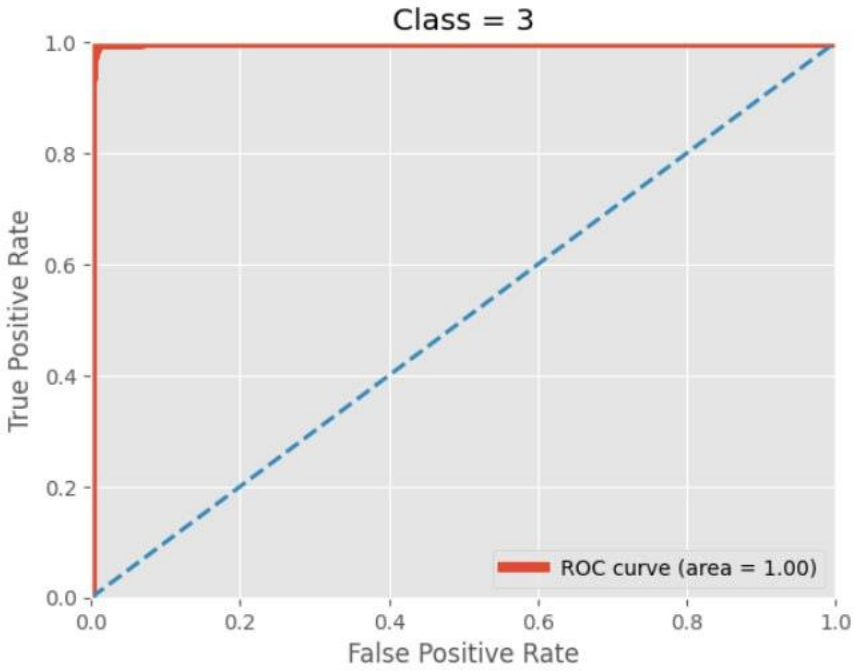


Figure 5.5: ROC Curve of Neural Network– 3

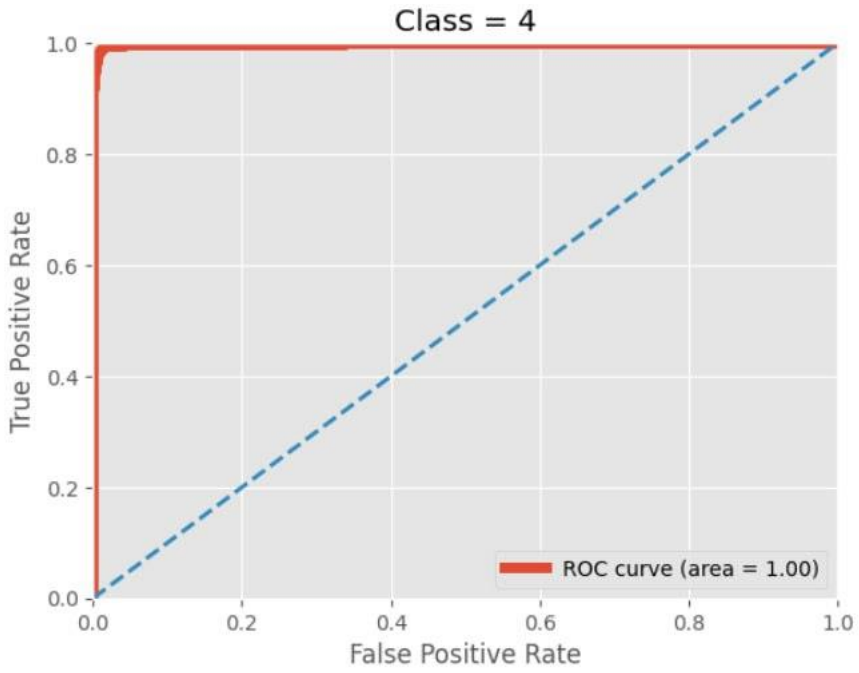


Figure 5.5: ROC Curve of Neural Network– 4

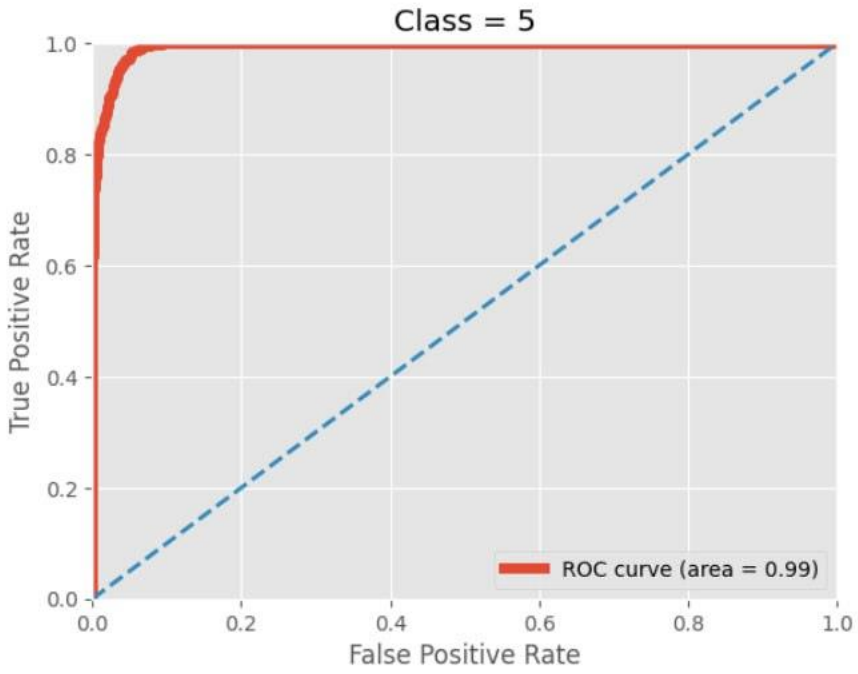


Figure 5.5: ROC Curve of Neural Network– 5

- The ROC curve for Class = 3 and Class = 4 shows An AUC (Area under the Curve) of 1, indicating excellent performance.

- True Positive Rate (Y-axis) vs. False Positive Rate (X-axis).
- A red curve indicating the Neural Network performance.
- A dashed blue line representing a random classifier.
- An AUC (Area under the Curve) of 0.99 for Class= 1 and Class 5, indicating excellent performance.
- An AUC (Area under the Curve) of 0.98 for Class= 2 indicating excellent performance.

Summary: The classifier is highly effective at distinguishing Class = 3 and Class=4 from other classes, with a very high AUC of 1.00, indicating maximal accuracy.

5.4 Final considerations

Our overall goals of models with pretty good metrics have been achieved with many different models. Furthermore, we had the chance to practice data and machine learning techniques and discover a lot of new technologies that are surely going to be useful in our future. We also had the chance to work with a real dataset and understand how to deal with it, from the data cleaning to the final model evaluation.